

**INSTITUTO FEDERAL DE EDUCAÇÃO CIÊNCIA E TECNOLOGIA DE SÃO
PAULO**

**CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS INTEGRADO AO
ENSINO MÉDIO**

SABOR E CLIC

Documentação do Projeto Integrador do Terceiro Bimestre

São Paulo

2026

SUMÁRIO

- 1 Apresentação do projeto
- 2 Objetivos e escopo
- 3 Concepção do sistema e integração entre disciplinas
- 4 Atores e responsabilidades
- 5 Fluxo geral do produto

6	Reserva
7	Sessão
8	Cardápio e carrinho
9	Pedidos
10	KDS
11	Administração de Bancadas e catálogo
12	Arquitetura da aplicação
13	Lógica JavaScript
14	Arquitetura Flask e rotas essenciais
15	Banco de dados e DER
16	Segurança e níveis de acesso
17	Identidade visual e prototipação
18	Requisitos funcionais e não funcionais
19	Tecnologias utilizadas
20	O que concluímos e próximas etapas
21	Atendimento aos requisitos do terceiro bimestre
	Referências

1 APRESENTAÇÃO DO PROJETO

Nós desenvolvemos o Sabor e Clic como uma proposta de sistema para organizar a operação de uma cozinha compartilhada com Bancadas reserváveis e atendimento gastronômico integrado. Nosso ponto de partida foi um problema recorrente: quando Reservas, disponibilidade de espaços, Cardápio e Pedidos são controlados por planilhas ou anotações isoladas, surgem conflitos de horário, falta de rastreabilidade e dificuldade para acompanhar a produção. Por isso, concebemos uma experiência única que liga o Cliente à cozinha e à administração.

Nossa documentação acompanha o fluxo do produto: Reserva → Sessão → Cardápio e Pedidos → KDS. Escolhemos esse encadeamento porque cada Pedido precisa existir dentro de um contexto de atendimento. A Reserva define quem utilizará qual Bancada e em qual período; a Sessão representa o uso efetivo dessa Reserva; o Cardápio permite selecionar itens; o Pedido registra a solicitação; e o KDS oferece à cozinha uma visão organizada da produção.

Ao longo do terceiro bimestre, reunimos contribuições de Artes Visuais e Design, JavaScript, Python, Banco de Dados e Segurança. Nosso protótipo estático demonstra as principais telas e interações. A modelagem de dados descreve as entidades e os relacionamentos necessários para sustentar a aplicação. Também iniciamos a estrutura de back-end e definimos a arquitetura teórica das rotas que completarão a integração.

2 OBJETIVOS E ESCOPO

2.1 Objetivo geral

Nosso objetivo é construir um sistema web que permita administrar Bancadas gastronômicas, criar Reservas, iniciar Sessões, consultar o Cardápio, enviar Pedidos e acompanhar seu preparo no KDS. Pretendemos centralizar dados que normalmente estariam dispersos e oferecer uma visão adequada para Cliente, Cozinheiro e Administrador.

2.2 Objetivos específicos

- Representar o fluxo completo do atendimento de forma clara e navegável.
- Modelar as entidades, atributos e relacionamentos em um DER e em um dicionário de dados.
- Aplicar JavaScript moderno para manipular o DOM, manter o carrinho e calcular valores em memória.
- Definir uma API Flask organizada por responsabilidades, com comunicação por JSON.
- Planejar autenticação, validação e RBAC para proteger ações e dados.
- Criar identidade visual coerente e interfaces para Cardápio Digital e KDS.

2.3 Limites desta etapa

Nesta etapa concluímos o protótipo estático e a documentação conceitual de dados. Iniciamos o back-end, mas a aplicação Flask integrada ao banco ainda faz parte da evolução

do projeto. Por esse motivo, quando descrevemos uma rota completa ou uma regra transacional, apresentamos a arquitetura que definimos para a implementação, distinguindo-a das interações que já funcionam localmente no protótipo.

3 CONCEPÇÃO DO SISTEMA E INTEGRAÇÃO ENTRE DISCIPLINAS

Nós organizamos o projeto como um sistema interdisciplinar. Design define como o usuário compreende estados e ações; JavaScript transforma a interface em uma experiência dinâmica; Flask centraliza regras e comunicação; Banco de Dados garante persistência e relacionamentos; Segurança controla identidade, permissão e validação. Nenhuma dessas partes resolve isoladamente o problema: a qualidade do fluxo depende da concordância entre todas elas.

Disciplina	Contribuição realizada	Evolução arquitetural
Design	Paleta, tipografia, componentes, telas por perfil, Cardápio e KDS.	Fidelidade ao protótipo, acessibilidade e testes de usabilidade.
JavaScript	Estado local, navegação, DOM, carrinho, cálculos e feedback visual.	Módulos, classes ES6 e integração assíncrona.
Python e Flask	Arquivos iniciais e definição da API.	MVC, controllers, services e rotas REST em JSON.
Banco de Dados	DER, MER, EER e dicionário de dados.	Modelo físico, scripts SQL, consultas e triggers.
Segurança	Definição de perfis e separação visual.	Autenticação, sessão, RBAC no servidor e sanitização.

4 ATORES E RESPONSABILIDADES

Ator	Objetivo no sistema	Principais ações
Cliente	Utilizar uma Bancada e consumir durante uma Sessão.	Consultar Bancadas, reservar, abrir Sessão, montar carrinho, enviar e acompanhar Pedidos.
Cozinheiro	Organizar a produção.	Consultar fila, iniciar preparo, marcar Pedido pronto e entregue, acompanhar tempos.
Administrador	Configurar e supervisionar a operação.	Gerenciar Bancadas, Reservas, Pratos, Ingredientes, usuários e relatórios.

Nós representamos os três atores por menus distintos. Essa separação melhora a compreensão do protótipo, mas sabemos que visibilidade de menu não equivale a autorização. Na aplicação integrada, cada requisição protegida precisará ser validada pelo Flask de acordo com o usuário autenticado e seu papel.

5 FLUXO GERAL DO PRODUTO

O fluxo começa quando o Cliente consulta uma data e um intervalo. O sistema compara capacidade e disponibilidade das Bancadas. Com uma opção válida, cria a Reserva e preserva o período e o valor acordado. No início do atendimento, a Reserva gera uma Sessão única. É a Sessão ativa que autoriza o acesso ao Cardápio e a criação de Pedidos. A cozinha recebe o Pedido no KDS, altera seu estado e devolve esse progresso ao Cliente. No encerramento, a Sessão deixa de aceitar novos Pedidos e conserva o histórico.

Etapa	Estado de entrada	Processamento	Saída
Reserva	Bancada disponível.	Validamos período, capacidade e conflito.	Reserva confirmada.
Sessão	Reserva válida.	Criamos contexto único e ativamos o atendimento.	Sessão ativa.
Cardápio	Sessão ativa.	Exibimos Pratos disponíveis e mantemos carrinho.	Itens selecionados.
Pedido	Carrinho válido.	Revalidamos itens, preços e quantidades.	Pedido recebido.
KDS	Pedido recebido.	Organizamos fila e controlamos transições.	Pedido em preparo, pronto e entregue.

Esse fluxo também orienta a estrutura de dados. O Pedido não precisa repetir Cliente ou Bancada, porque sua Sessão conduz à Reserva e a Reserva identifica ambos. Essa decisão reduz redundância e mantém uma única fonte para o contexto do atendimento.

6 RESERVA

6.1 Lógica de negócio

Nós tratamos a Reserva como o compromisso de uso de uma Bancada durante um intervalo. Para criar uma Reserva, precisamos identificar o Cliente, a Bancada, o início, o fim, o número de pessoas e o valor. A Bancada deve estar ativa, comportar o grupo e não possuir outra Reserva incompatível. O fim precisa ser posterior ao início.

A condição abstrata de conflito é: há sobreposição quando o início solicitado ocorre antes do fim existente e o fim solicitado ocorre depois do início existente. Essa regra deve ser executada no servidor imediatamente antes da gravação. A tela pode antecipar a

disponibilidade, mas somente o back-end pode confirmar, pois outra pessoa pode reservar o mesmo intervalo enquanto o primeiro Cliente ainda preenche o formulário.

6.2 Contribuição de cada disciplina

Em Design, criamos cartões de Bancada, etiquetas de disponibilidade, filtros e formulário de Reserva. Em JavaScript, utilizamos o array benches e templates para renderizar capacidade, recursos, preço e estado. Em Banco de Dados, relacionamos CLIENTE e BANCADA à RESERVA. Em Flask, definimos a necessidade de rotas de consulta, criação, alteração e cancelamento. Em Segurança, estabelecemos que o Cliente só pode manipular suas próprias Reservas, enquanto o Administrador possui acesso operacional mais amplo.

```
const benches = [  
  {id:1, nome:"Bancada Toscana", cap:4, preco:85,  
    status:"Disponível", recursos:["Cooktop", "Pia", "Tomadas"]}  
];
```

Trecho 1 - Estrutura real utilizada para representar Bancadas no protótipo.

No protótipo, os dados permanecem em memória e os botões simulam confirmação. Na aplicação final, o JavaScript enviará os campos em JSON, exibirá estado de carregamento e tratará respostas 201, 400 ou 409 sem recarregar a página.

7 SESSÃO

Nós adotamos Sessão como entidade central do atendimento. Uma Reserva descreve o período contratado; a Sessão registra o uso efetivo e reúne os Pedidos daquele período. Cada Reserva deve gerar exatamente uma Sessão, protegida por uma relação um para um. A Sessão possui código de acesso, estado e datas de criação, início e encerramento.

Uma Sessão pode estar criada, ativa ou encerrada. Somente ativa aceita Pedidos. A ativação registra o início real; o encerramento invalida o código de acesso e bloqueia novas solicitações. Essa verificação precisa acontecer no Flask a cada Pedido, pois não podemos confiar apenas no botão da interface.

Na tela que desenvolvemos, mostramos tempo restante, quantidade de Pedidos, consumo, pessoas, Pedidos vinculados e ações de navegação. Em JavaScript, essa tela é uma função do objeto views. No banco, SESSAO recebe uma referência única à RESERVA e origina vários PEDIDOS. Em Segurança, o acesso deve ser limitado ao Cliente responsável, à operação autorizada e ao Administrador.

8 CARDÁPIO E CARRINHO

Nosso Cardápio Digital apresenta foto, nome, descrição, categoria, preço e disponibilidade. Criamos oito produtos demonstrativos e organizamos a experiência em Entradas, Pratos, Bebidas e Sobremesas. Quando um item está indisponível, desabilitamos seu botão de adição. Essa regra visual ajuda o usuário, mas o servidor deverá revalidá-la na confirmação.

8.1 Lógica do carrinho

```
function addCart(id){
  const p=products.find(x=>x.id===id);
  if(p.status!=="Disponível") return;
  const ex=state.cart.find(x=>x.id===id);
  if(ex) ex.qtd++;
  else state.cart.push({id:p.id,nome:p.nome,preco:p.preco,qtd:1});
}
function cartTotal(){
  return state.cart.reduce((s,i)=>s+i.preco*i.qtd,0);
}
```

Trecho 2 - Adição de item e cálculo do total no protótipo.

Primeiro localizamos o produto pelo identificador. Se ele estiver indisponível, encerramos a função. Depois procuramos o mesmo item no carrinho: quando existe, incrementamos a quantidade; quando não existe, adicionamos um novo objeto. O total é calculado com reduce, acumulando preço multiplicado pela quantidade. Essa lógica satisfaz o objetivo do terceiro bimestre de manipular arrays, objetos e cálculos em memória.

Na integração, o navegador enviará identificadores e quantidades, mas não será responsável pelo valor final. O Flask consultará os preços atuais, validará cada Prato e calculará os subtotais. Isso evita que um preço alterado no console do navegador seja aceito como verdadeiro.

9 PEDIDOS

O Pedido agrupa itens solicitados durante uma Sessão. Definimos que ele deve possuir ao menos um ITEM_PEDIDO, estado, valor total, tempo estimado e data de criação. ITEM_PEDIDO preserva quantidade, preço unitário praticado e subtotal. Mesmo que o preço do catálogo mude posteriormente, o histórico continua representando a compra original.

Ao confirmar, o servidor deverá verificar autenticação, propriedade da Sessão, estado ativo, lista não vazia, quantidades positivas e Pratos disponíveis. Em seguida, buscará os preços, calculará subtotais, gravará Pedido e itens na mesma transação e devolverá JSON. Se qualquer etapa falhar, a transação inteira deve ser desfeita.

Estado	Significado	Próxima ação esperada
AGUARDANDO ou RECEBIDO	Pedido aceito e visível para a cozinha.	Iniciar preparo ou cancelar conforme regra.
EM PREPARO	Produção iniciada.	Marcar como pronto.
PRONTO	Pedido finalizado pela cozinha.	Marcar como entregue.
ENTREGUE	Ciclo operacional concluído.	Somente consulta histórica.
CANCELADO	Pedido interrompido.	Somente consulta ou procedimento administrativo.

10 KDS

O Kitchen Display System é o painel do Cozinheiro. Nós o organizamos em colunas para tornar o estado imediatamente visível: Recebidos, Em preparo e Prontos. Cada ticket apresenta número, Bancada, tempo e itens. Cores e etiquetas reforçam prioridade e atraso. Essa estrutura reduz a necessidade de abrir cada Pedido separadamente.

No protótipo, os tickets são exemplos fixos e os botões acionam um toast. Na arquitetura definida, o KDS consultará a API e enviará alterações de estado. A interface só moverá o ticket depois da confirmação do servidor. O Flask validará a transição para impedir saltos ou regressões incoerentes. Dois Cozinheiros atuando ao mesmo tempo deverão receber uma resposta de conflito quando tentarem atualizar uma versão antiga do Pedido.

Design contribui com hierarquia, cores e leitura rápida. JavaScript controla renderização e atualização sem F5. Flask organiza fila e transições. Banco registra estado e horários. Segurança limita as ações a Cozinheiro e Administrador. Assim, o KDS reúne todas as disciplinas em uma única função operacional.

11 ADMINISTRAÇÃO DE BANCADAS E CATÁLOGO

O Administrador configura os recursos utilizados nos fluxos anteriores. Desenvolvemos telas para Bancadas, Reservas, Produtos, Ingredientes, substituições, usuários

e relatórios. As operações exibidas ainda são demonstrativas, mas definem a experiência e os dados necessários para os futuros CRUDs.

A exclusão de Prato deve ser lógica por meio do atributo ativo. Dessa forma, um item deixa de aparecer para novas compras sem destruir ITEM_PEDIDO histórico. A ficha técnica relaciona Prato e Ingrediente com quantidade necessária. A substituição é direcional e possui prioridade e fator de conversão. A futura disponibilidade poderá somar o consumo de todos os itens e escolher substitutos autorizados quando o ingrediente original estiver insuficiente.

12 ARQUITETURA DA APLICAÇÃO

Nós definimos uma arquitetura em camadas. O navegador contém HTML, CSS e JavaScript; a comunicação utiliza HTTP e JSON; o Flask recebe requisições, autentica, valida e executa regras; a camada de dados persiste entidades relacionais. O banco nunca deve ser acessado diretamente pelo navegador.

Camada	Responsabilidade	Exemplo
View	Apresentar conteúdo e capturar eventos.	Cardápio, formulário e KDS.
JavaScript de página	Gerenciar estado visual e chamar API.	Carrinho, filtros e feedback.
Routes Flask	Interpretar HTTP, parâmetros e códigos de resposta.	POST /api/reservas.
Controllers e services	Executar caso de uso e regras.	Verificar conflito e criar Sessão.
Models e repositórios	Representar e consultar dados.	Reserva, Sessão, Pedido e Prato.
Banco relacional	Garantir persistência e integridade.	PK, FK, unicidade e transações.

O README registra uma estrutura futura modular, com routes, controllers, models, services e middleware. Os arquivos atuais de back-end são iniciais e ainda não formam essa aplicação completa. Documentamos a estrutura como direção arquitetural que adotamos, sem afirmar que todos os módulos já estão implementados.

13 LÓGICA JAVASCRIPT

O objeto state concentra perfil, página atual, carrinho e estado de Reserva. menus associa cada papel às telas permitidas; roleInfo guarda contas demonstrativas; views reúne funções que devolvem templates HTML. A função render substitui o conteúdo do elemento

app e renderMenu reconstrói a navegação. Com isso, simulamos uma SPA simples sem biblioteca externa.

```
function go(page){
  state.page=page;
  renderMenu();
  render();
  window.scrollTo(0,0);
}
function render(){
  pageTitle();
  el("app").innerHTML=views[state.page]();
}
```

Trecho 3 - Navegação e renderização dinâmica do protótipo.

Utilizamos const, arrow functions, template literals, map, filter, reduce, find e for...in. Também criamos funções auxiliares para cartões, indicadores e tickets. showToast fornece feedback temporário ao usuário. O login demonstra a seleção de perfil por e-mail, porém qualquer senha é aceita; portanto, ele representa navegação por papéis e não autenticação de produção.

Na próxima etapa, pretendemos separar modelos, API e páginas. Classes como Prato, ItemPedido, Carrinho e Pedido poderão encapsular estado e validações. As chamadas fetch serão assíncronas e envolvidas em try...catch, com estados de carregamento, sucesso e erro. Dados externos não deverão ser inseridos por innerHTML sem escape, para reduzir risco de XSS.

14 ARQUITETURA FLASK E ROTAS ESSENCIAIS

As rotas a seguir representam a arquitetura que definimos para completar o produto. Nosso back-end atual possui arquivos iniciais, mas a API ainda deverá ser organizada e integrada. Em MVC, a rota recebe HTTP; o controller coordena o caso de uso; o service aplica regras; o model acessa dados; e a rota devolve JSON com código adequado.

Método e rota	Atores	Lógica essencial
POST /api/auth/login	Todos	Validar credenciais, usuário ativo, criar Sessão de autenticação e retornar perfil.
GET /api/bancadas	Cliente/Admin	Filtrar capacidade e ausência de conflito no intervalo.
POST /api/reservas	Cliente/Admin	Revalidar disponibilidade em transação, calcular valor e criar Reserva.
POST /api/reservas/{id}/sessao	Cliente/Admin	Garantir Reserva válida e Sessão única; gerar código de acesso.
GET /api/cardapio	Cliente	Listar Pratos ativos e disponíveis.

Método e rota	Atores	Lógica essencial
POST /api/sesoes/{id}/pedidos	Cliente	Validar Sessão e itens; recalcular preços; gravar Pedido e itens.
GET /api/kds/pedidos	Cozinheiro/Admin	Listar fila por estado e tempo.
PATCH /api/pedidos/{id}/status	Cozinheiro/Admin	Validar transição, atualizar estado e horários.
POST /api/sesoes/{id}/encerrar	Cliente/Admin	Bloquear novos Pedidos e registrar encerramento.

14.1 Exemplo teórico de rota

```
@bp.post("/api/sesoes/<int:id>/pedidos")
@login_required
def criar_pedido(id):
    payload = request.get_json() or {}
    pedido = pedido_service.criar(
        sessao_id=id, usuario=g.user, itens=payload.get("itens", [])
    )
    return jsonify(pedido.to_dict()), 201
```

Trecho 4 - Exemplo teórico da responsabilidade reduzida de uma rota Flask.

Nesse exemplo, a rota não calcula preços nem consulta estoque diretamente. Ela autentica por decorador, lê JSON, entrega dados ao serviço e serializa o resultado. O service confirma a propriedade da Sessão, verifica se está ativa, valida quantidades, busca Pratos, calcula valores e grava a transação. Essa divisão evita controllers extensos e facilita testes.

Utilizaremos 200 para leituras e alterações bem-sucedidas, 201 para criação, 400 para dados inválidos, 401 para falta de autenticação, 403 para papel inadequado, 404 para recurso inexistente e 409 para conflito de Reserva, Sessão ou estado. O JavaScript interpretará esses códigos e mostrará feedback específico.

15 BANCO DE DADOS E DER

Nós modelamos USUARIO e suas especializações CLIENTE, COZINHEIRO e ADMINISTRADOR; BANCADA e RESERVA; SESSAO, PEDIDO e ITEM_PEDIDO; PRATO e INGREDIENTE; além de FICHA_TECNICA e SUBSTITUIDO_POR. O dicionário descreve atributos, tipos sugeridos, domínios e regras de consistência.

Figura 1 - Diagrama Entidade Relacionamento do Sabor e Clic. Fonte: elaboração da equipe, 2026.

A cadeia RESERVA ↔ SESSAO → PEDIDO → ITEM_PEDIDO é o núcleo do modelo. CLIENTE solicita RESERVA e BANCADA é utilizada pela RESERVA. Cada item

referencia um PRATO. FICHA_TECNICA resolve a relação muitos para muitos entre Prato e Ingrediente, enquanto SUBSTITUIDO_POR utiliza duas referências para Ingrediente.

Figura 2 - Modelo de entidades e relacionamentos. Fonte: elaboração da equipe, 2026.

Definimos regras de integridade: e-mail único, valores não negativos, fim posterior ao início, Sessão única por Reserva, quantidade positiva, Pedido com itens e chaves estrangeiras obrigatórias. No quarto bimestre, o modelo físico deverá traduzir essas regras para SQL, constraints, índices, inserções básicas, consultas e triggers de estoque.

16 SEGURANÇA E NÍVEIS DE ACESSO

Definimos RBAC com três papéis. Cliente acessa seus próprios dados; Cozinheiro opera Pedidos e KDS; Administrador gerencia cadastros e visão geral. No protótipo, essa divisão aparece nos menus. Na aplicação final, ela será aplicada no servidor por decorators ou middleware, porque ocultar um botão não impede uma chamada HTTP direta.

Risco	Decisão arquitetural
Senha exposta	Armazenar hash forte com salt; nunca senha em texto ou cifra reversível.
Preço manipulado	Recalcular total no Flask a partir do catálogo.
Reserva duplicada	Validar sobreposição dentro de transação.
SQL Injection	Utilizar ORM ou consultas parametrizadas.
XSS	Escapar dados e preferir textContent para conteúdo externo.
CSRF/cookie	Configurar HttpOnly, Secure, SameSite e proteção CSRF quando aplicável.
Acesso indevido	Verificar identidade, propriedade do recurso e papel em cada rota.

O utilitário Fernet existente pode servir para dados que precisem ser recuperados, mas não para senha. Para senha, adotaremos hash unidirecional. Também planejamos logs sem informações sensíveis, mensagens genéricas para falhas de login e validação duplicada no cliente e no servidor.

17 IDENTIDADE VISUAL E PROTOTIPAÇÃO

Nós construímos uma identidade inspirada em materiais e cores gastronômicas. Utilizamos grafite #2B241E para estrutura, fundo #F6F1E8, cobre #BB6B2F como destaque,

verde #4B7A52 para sucesso, açafrão #B9832A para atenção e páprica #A6432E para erro. Archivo aparece em títulos e Hanken Grotesk nos elementos de interface.

Criamos componentes consistentes: menu lateral, cartões, badges, botões, tabelas, filtros, indicadores, timeline e colunas do KDS. CSS Grid e Flexbox organizam o conteúdo, enquanto media queries adaptam a interface. As imagens das Bancadas e dos produtos aproximam o protótipo de uma experiência de uso real.

Ainda precisamos incorporar ao repositório os wireframes e o link/exportação do protótipo Figma. O front-end atual funciona como protótipo navegável codificado e demonstra telas de Cliente, Cozinheiro e Administrador.

[INSERIR FIGURA - WIREFRAMES E PROTÓTIPO FIGMA]

18 REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS

Código	Requisito funcional	Estágio
RF01	Entrar e acessar interface correspondente ao papel.	Demonstrado no protótipo; autenticação real é próxima etapa.
RF02	Consultar e reservar Bancadas.	Interface concluída; persistência planejada.
RF03	Criar, ativar e encerrar Sessão.	Modelado e prototipado.
RF04	Consultar Cardápio e manter carrinho.	Implementado em memória.
RF05	Criar e acompanhar Pedido.	Interface concluída; API planejada.
RF06	Operar fila no KDS.	Layout concluído; atualização persistente planejada.
RF07	Administrar catálogo, Ingredientes e usuários.	Telas demonstrativas concluídas; CRUD planejado.

Requisito não funcional	Como tratamos
Usabilidade	Hierarquia, feedback, navegação por perfil e consistência visual.
Responsividade	Grid, Flexbox e media queries no CSS.
Manutenibilidade	Planejamento modular de pages, models, api, routes e services.
Segurança	RBAC, validação no servidor, hash e consultas parametrizadas.
Integridade	Transações, PK/FK, unicidade e domínios de estado.
Desempenho	Filtros no servidor, índices futuros e atualização parcial do DOM.

19 TECNOLOGIAS UTILIZADAS

Tecnologia	Aplicação no projeto
HTML5	Estrutura das páginas e formulários.
CSS3	Identidade visual, componentes e responsividade.
JavaScript ES6+	DOM, estado, arrays, templates, carrinho e cálculos.
JSON e Fetch API	Formato e mecanismo definidos para integração; exemplo inicial no back-end.
Python e Flask	Base escolhida para API, regras e autenticação.
Banco relacional/MySQL Workbench	DER, MER, EER e futura persistência SQL.
Git e GitHub	Versionamento e integração do trabalho da equipe.

20 O QUE CONCLUÍMOS E PRÓXIMAS ETAPAS

20.1 O que concluímos

Concluímos o protótipo estático navegável, a identidade visual, as telas dos três atores, o Cardápio, o carrinho em memória, a visualização de Pedidos, o layout do KDS, o DER, o MER, o arquivo EER e o dicionário de dados. Também definimos o fluxo Reserva → Sessão → Cardápio/Pedidos → KDS e a responsabilidade esperada de cada camada.

20.2 O que iniciamos

Iniciamos a estrutura Python, a criação do banco e um exemplo de comunicação Fetch/JSON. Também iniciamos a definição de segurança e separação de perfis. Essas partes ainda não formam uma API integrada, mas registram a direção que seguiremos.

20.3 Próximas etapas

Nas próximas etapas, implementaremos a factory Flask, os módulos MVC, autenticação e RBAC, as rotas de Bancada e Reserva, a criação de Sessão, Cardápio e Pedido, o KDS persistente e o banco físico. Depois integraremos fetch ao front-end, trataremos loading/erros e criaremos testes para conflitos, permissões, totais e transições de estado.

21 ATENDIMENTO AOS REQUISITOS DO TERCEIRO BIMESTRE

Requisito	Situação	Evidência produzida
DER	Concluído	DER_saborclic.png,

Requisito	Situação	Evidência produzida
		MER_saborclic.png e EER_saborclic.mwb.
Dicionário de dados	Concluído	Documento em DOCX e PDF com entidades, atributos e relações.
Wireframes/Figma	Em complementação	Protótipo codificado existe; arquivo ou link Figma deverá ser incluído.
Identidade visual	Concluída	Paleta, tipografia, marca e componentes em CSS.
Cardápio Digital	Concluído como protótipo	Produtos, categorias, imagens, disponibilidade e carrinho.
KDS	Concluído como protótipo	Colunas, tickets, tempos e ações simuladas.
JavaScript/DOM	Concluído para a etapa	Renderização dinâmica, eventos, arrays, map/filter/reduce e cálculo.
Validações	Parcial	required e regras locais; validações completas serão repetidas no Flask.
Flask/MVC/rotas/JSON	Estrutura teórica e início técnico	Arquivos iniciais, exemplo fetch e arquitetura detalhada neste documento.
Segurança e RBAC	Definido conceitualmente	Papéis e menus; middleware e autenticação serão implementados.

REFERÊNCIAS

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO. Projeto Integrador Sabor & Clic: sistema de gestão para dark kitchens e espaços gastronômicos. Versão de 31 de agosto de 2026. São Paulo: IFSP, 2026.

SABOR E CLIC. Repositório Sabor-Clic. GitHub, 2026. Disponível em: <https://github.com/FaellEmiliano/Sabor-Clic>.

SABOR E CLIC. Dicionário de Dados Sabor e Clic. São Paulo, 2026.

ATUALIZAÇÃO DA IMPLEMENTAÇÃO — 3º BIMESTRE

Esta seção atualiza a documentação original com o estado mais recente do projeto, integrando o protótipo estático do front-end, os scripts JavaScript exigidos no terceiro bimestre e a estrutura inicial do back-end Flask. O objetivo desta etapa continua sendo demonstrar arquitetura, fluxos e regras fundamentais com dados simulados, sem antecipar a integração completa prevista para o quarto bimestre.

1. Visão integrada da implementação atual

A aplicação está organizada em três camadas principais: interface HTML/CSS, lógica de interação em JavaScript e estrutura inicial de back-end em Flask.

No estado atual, o front-end utiliza dados simulados e armazenamento local para demonstrar os fluxos. O back-end define contratos de endpoints e modelos simulados, ainda sem persistência definitiva no banco de dados.

A arquitetura definitiva manterá o mesmo fluxo conceitual: Reserva → Sessão → Cardápio/Carrinho → Pedido → KDS. A integração real entre JavaScript e Flask será feita por Fetch API e JSON no quarto bimestre.

2. Front-end atual

O front-end está estruturado como uma aplicação multipágina (MPA), com páginas separadas por perfil de usuário. As telas usam os mesmos componentes visuais e dados simulados para manter consistência entre cliente, cozinha e administração.

- Acesso: index.html (login simulado).
- Cliente: início, bancadas, reservas, sessão, cardápio, carrinho e pedidos.
- Cozinha: dashboard, KDS, pedidos, disponibilidade do cardápio/estoque e histórico.
- Administrador: dashboard, reservas, bancadas, produtos, ingredientes, substituições, usuários e relatórios.

Essas páginas funcionam como protótipo navegável da aplicação final e permitem demonstrar os principais fluxos sem depender do banco de dados ou do servidor Flask.

3. JavaScript — lógica do 3º bimestre

A lógica JavaScript foi separada por responsabilidade para evitar um arquivo monolítico e tornar explícitos os conteúdos avaliados no terceiro bimestre.

Arquivo	Responsabilidade	Conteúdos demonstrados
data.js	Dados simulados e estado compartilhado	arrays, objetos literais, for...in
site.js	Navegação, sessão simulada e helpers gerais	DOM, localStorage, eventos
components.js	Componentes visuais reutilizáveis	for...of, templates e composição de interface
catalog.js	Renderização dinâmica e filtros do cardápio	forEach, arrays e DOM
cart.js	Carrinho e cálculos em memória	find, filter, reduce, forEach, subtotal e total
validation.js	Validação da reserva	for...of, validação de entradas e feedback ao usuário
login.js	Login de demonstração	validação de formulário e seleção de perfil
views.js	Registro das telas do protótipo	renderização das páginas e dados simulados
a11y.js	Acessibilidade de interface	comportamentos auxiliares e navegação acessível

Nesta etapa, Fetch API, async/await, tratamento de erros HTTP e classes ES6+ não são necessários para a entrega. Esses recursos serão utilizados na integração real com o back-end no quarto bimestre.

4. Fluxos funcionais demonstrados no front-end

- Cardápio: produtos armazenados em arrays de objetos são renderizados dinamicamente no DOM e podem ser filtrados por categoria.
- Carrinho: itens podem ser adicionados e removidos; quantidades, subtotais e valor total são calculados em memória.
- Reserva: o formulário executa validações simples antes de apresentar confirmação simulada.
- Perfis: o login utiliza contas de demonstração e localStorage para direcionar cliente, cozinheiro e administrador às respectivas áreas.
- KDS: a interface demonstra visualmente a fila de pedidos e seus estados, ainda sem sincronização real com o servidor.

5. Back-end Flask simulado

O back-end do terceiro bimestre tem caráter estrutural. Ele demonstra como a aplicação Flask será organizada e quais endpoints fundamentais existirão, mas utiliza modelos simulados e ainda não realiza persistência definitiva.

- run.py: ponto de inicialização do servidor Flask.
- config.py: concentra configurações principais da aplicação.
- app/__init__.py: criação/configuração inicial da aplicação.
- controller/auth_controller.py: exemplos de cadastro, consulta de usuário e login.
- controller/reserva_controller.py: consulta de bancadas e criação/consulta de reservas.
- controller/pedidos_controller.py: cardápio, criação/consulta de pedidos e consulta do KDS.

6. Endpoints fundamentais definidos

Método	Endpoint	Finalidade	Estado atual
POST	/api/cadastro	Cadastrar cliente	Simulado
GET	/api/cadastro/<id>	Consultar usuário por ID	Simulado
POST	/api/login	Validar login	Simulado
GET	/api/bancadas	Listar	Simulado

		bancadas	
POST	/api/reservas	Criar reserva	Simulado
GET	/api/ reservas/<id>	Consultar reserva	Simulado
GET	/api/cardapio	Obter cardápio	Simulado
POST	/api/pedido/ cadastro	Criar pedido vinculado à reserva	Simulado
GET	/api/pedido/ <id>	Consultar pedido	Simulado
GET	/api/kds/<id>	Obter representação do pedido para o KDS	Simulado

7. Integração entre os sistemas

Mesmo com front-end e back-end ainda simulados, os dois lados foram organizados para representar o mesmo domínio. O front-end já apresenta as telas e ações que futuramente chamarão os endpoints Flask, enquanto o back-end define os contratos básicos correspondentes.

1. O cliente seleciona uma bancada e informa os dados da reserva.
2. A reserva representa o vínculo entre cliente, bancada e período de uso.
3. A reserva dá acesso a uma Sessão, que será o contexto de uso durante o período reservado.
4. Na Sessão, o cliente consulta o cardápio e monta o carrinho.
5. Ao finalizar, o pedido é criado associado à reserva/sessão.
6. O pedido passa a ser exibido no KDS do cozinheiro.
7. No sistema completo, alterações de estado feitas no KDS serão persistidas no back-end e refletidas no front-end.

8. Contratos previstos para a integração no 4º bimestre

A implementação atual evita criar uma API descartável. Os dados simulados já seguem uma estrutura próxima da versão final, permitindo substituir gradualmente os mocks por requisições HTTP.

- Reserva: cliente, bancada, data, horário e quantidade de pessoas.
- Pedido: identificador da reserva/sessão e lista de itens com quantidade.
- Cardápio: identificador, nome, categoria, descrição, preço e disponibilidade.
- KDS: pedido, itens, bancada/sessão, horário e estado de preparo.

9. Segurança na etapa atual

O login e o controle de perfis atuais têm função exclusivamente demonstrativa e utilizam localStorage. Eles não representam autenticação segura. No quarto bimestre, a autenticação será transferida para Flask, com sessões/cookies, controle de acesso por perfil (RBAC), validação também no servidor e consultas parametrizadas ao banco.

10. Atendimento aos requisitos do 3º bimestre

Requisito	Situação	Evidência
DER conceitual	Concluído	Documentação existente e arquivo do brModelo
Dicionário de dados	Concluído	Documento específico na pasta Documentação
Protótipo e páginas principais	Concluído	FrontEnd com áreas de cliente, cozinha e administrador
Cardápio digital	Concluído como protótipo funcional	cliente_cardapio.html + catalog.js
KDS	Concluído como protótipo visual	cozinha_kds.html / views.js
Arrays e objetos	Concluído	data.js

literais		
for...of / for...in / forEach	Concluído	components.js, data.js, catalog.js/cart.js
Manipulação dinâmica do DOM	Concluído	catalog.js e scripts de interface
Validação de formulários	Concluído em nível inicial	validation.js e login.js
Cálculos em memória	Concluído	cart.js
Estrutura Flask	Concluída em nível inicial	run.py, config.py e app
Endpoints fundamentais	Concluídos como exemplos simulados	controllers do back- end
Integração JS ↔ Flask	Planejada para o 4º bimestre	Fetch/JSON ainda não conectados

11. Estado da primeira entrega e próximos passos

Ao final do terceiro bimestre, o projeto possui modelagem de dados, identidade e protótipos das interfaces, uma aplicação front-end navegável com lógicas JavaScript fundamentais e uma estrutura de back-end Flask que define os principais contratos do sistema. A etapa seguinte substituirá progressivamente os dados simulados por persistência real, integrará o front-end ao Flask por Fetch API e aplicará autenticação, autorização e regras de negócio completas.